Implementation Plan: First SAFS Dynamic-Rupture Production Run

Date: 2026-05-18 **Branch:** `feature/safs-quasi-dynamic` **Driver:** `seas_spatial_dyn_driver` (`drivers/spatial_dyn_driver.cpp`)

Overview

End-to-end enablement of the first SAFS dynamic-rupture production run through `seas_spatial_dyn_driver`. The target configuration is constant-material (TPV205- style `λ = μ = 32 GPa`, `ρ = 2670 kg/m³`); regional CSM stress projected onto the fault trace via the existing `ApplyCsmStressSidecar` path; LSW friction with `geoffrey2010.md` parameters; and a single new nucleation mechanism named `gradual_overstress` (per-DOF, smoothStep-ramped, Gaussian-shaped δτ accumulator on `DOFData[i].tau1_nuc / tau2_nuc`). The plan covers four work streams:

- **Phase N** — replace the stubbed `Overstress` / forced-rupture `StrengthReduction` nucleation paths in the spatial driver with the single new `gradual_overstress` kind, including a per-sub-step accumulator hook in `Tpv205SubStepIterator`.
- **Phase D** — implement a real `--print-derived` pre-flight that aborts on ill-posed initial conditions (the gate that prevents wasting Frontera time).
- **Parity Phases 1-6** — adopt the full ParaView paraview-compaction feature surface as enumerated in `safs/project_7.0_alternative/debug_document/spatial_paraview_compaction_parity_plan_2026-05-18.md` (CLI flag surface, build guards, default-OFF master gate, secondary stress collection, volume velocity, SAFS fault static fields including `nuc_amplitude` / `nuc_radial_factor`).
- **Phase Z** — Frontera sbatches (smoke + production) and a `fault.vtkhdf` post-run verifier.

The TPV* and BP5 byte-exact regression contract MUST stay green at every commit.

Reference materials (read-only during implementation)

- `safs/project_7.0_alternative/document/spatial_dynamic_rupture_plan.md` — parent plan (already updated for `gradual_overstress`).
- `safs/project_7.0_alternative/document/spatial_friction_config_schema.md` — TOML schema source of truth (already updated for `gradual_overstress`).
- `safs/project_7.0_alternative/debug_document/spatial_paraview_compaction_parity_plan_2026-05-18.md` — ParaView parity plan (already updated for `nuc_amplitude` / `nuc_radial_factor`).
- `safs/project_7.0_alternative/debug_document/spatial_workflow_safs_runbook_2026-05-18.md` — user-facing runbook.
- `miniapps/seas/CLAUDE.md` — numerical conventions, ParaView output mode policy, spatial-driver nucleation subsection.
- `miniapps/seas/dynamic/tpv104_nucleation.hpp` — implementation source for the temporal smoothStep helpers; **rename them generic, do not import as-is**.
- `miniapps/seas/dynamic/tpv104_substep_iterator.{hpp,cpp}` — reference for the per-sub-step nucleation hook pattern.
- `miniapps/seas/dynamic/tpv205_substep_iterator.{hpp,cpp}` — current TPV205 iterator (no nucleation hook today; we add one).
- `miniapps/seas/drivers/tpv104_driver.cpp` — reference for full ParaView wiring patterns + AdaptiveSchedule usage.
- `miniapps/seas/io/paraview_output.hpp` — `seas::ParaViewOutput`, `SetFaultParamsBP5`, `SetTotalRunTime`, `AdaptiveSchedule`, `RegisterDomainField`, `SetVolumeHDFCompression`, `SetFaultHDFCompression`.
- `miniapps/seas/dynamic/spatial_setup.hpp` — `InitializeFaultDOFs_Spatial` (already zeroes `tau1_nuc / tau2_nuc / sigma_n_nuc` at L88, L134).

Constraints

Interface constraints — what cannot change

- **`FaultFaceFlux::DOFData` field layout** (`dynamic/fault_face_flux.hpp:43-46`) — `tau1_nuc` (dip), `tau2_nuc` (strike), `sigma_n_nuc` are pre-existing real_t fields. The `gradual_overstress` accumulator writes

into them; do NOT add new DOFData fields.
- **`InitializeFaultDOFs_Spatial` semantics** — already zeroes `tau1_nuc / tau2_nuc / sigma_n_nuc` (line 88, 134). The new accumulator contract relies on this; do not change that initialization.
- **Native TPV* / BP5 nucleation paths** — `dynamic/tpv104_nucleation.hpp` (`SmoothStep_TPV104` / `ApplyNucleationIncremental_TPV104`), `Tpv104SubStepIterator` internal calls at `tpv104_substep_iterator.cpp:295, 627`, and the `FaultFrictionLaw::LSW_ForcedRupture` enum + `EvaluateADER_LSW_ForcedRupture` flux + `T_forced_rupture` / `t0_decay_forced` DOFData fields ALL stay in place. The native TPV104 driver may still use the smoothStep helpers; the native TPV205 driver may still reference `LSW_ForcedRupture` if it ever needs forced rupture. Only the spatial driver's enum value, resolver, and dispatch into those paths are removed.
- **`ParaViewOutput<ParMesh>` API surface** as documented in `io/paraview_output.hpp` — `RegisterDomainField`, `InitFaultOutputBP5`, `SetFaultParamsBP5`, `SetFaultOutputMode`, `SetVolumeHDFCompression`, `SetFaultHDFCompression`, `SetTotalRunTime`, `GetSchedule`, `fixed_dt`, `output_every_n_steps`. Phase 6 adds ONE new method `SetFaultParamsSpatial(...)`; everything else is consumed verbatim.
- **`Tpv205SubStepIterator::AdvanceWithSubStepStates(...)`** signature (`dynamic/tpv205_substep_iterator.hpp:102-110`) — exposed by Phase N as a **purely additive callback overload** (a second method that takes the callback; the original signature is preserved untouched for any test that may already exist).
- **Sign conventions** per `miniapps/seas/CLAUDE.md` "Critical Numerical Details" — `σ_n > 0 = compression`; dip direction `(0, 0, +1)` downward; fault tangent frame `t1 = dip, t2 = strike` (Tandem `FaultBasis` convention); `τ_pre  V_init`. The `gradual_overstress` accumulator writes `Δτ_dip → tau1_nuc` and `Δτ_strike → tau2_nuc` consistent with this frame.

Dependency constraints

- **`MFEM_USE_MPI = YES`** (driver hard-requires it at `spatial_dyn_driver.cpp:682`).
- **`MFEM_USE_HDF5 = YES`** for VTKHDF; runtime `HDF5_PLUGIN_PATH` set for `MFEM_USE_H5Z_ZFP` (ZFP defaults).
- **`SEAS_USE_TOML`** defined when `extern/toml11/toml.hpp` is present (auto by Makefile).
- **`paraview-compaction` API** already merged into this branch (commits `240581e` and `0ce73a4`).
- **No new external libraries.**

Convention constraints

- **Greppable markers** — every TODO this plan defers tags `// SPATIAL-DYN-FIRST-RUN R-XXX TODO`. `R-XXX` numbering starts at `R-001` in this plan.
- **TOML is the source of truth; CLI overrides win.** Every CLI flag added in Parity Phase 1 has a matching `OutputSpec` field added in Parity Phase 2 and a matching schema-doc row.
- **Naming** — new source files under `dynamic/spatial_nucleation.{hpp,cpp}` (mirrors `dynamic/spatial_setup.hpp`); new test `tests/unit/test_spatial_nucleation.cpp`; new sbatches under `jobs/safs/`; new verifier under `safs/project_7.0_alternative/spatial/code/scripts/`.
- **No hardcoded numerical constants** — every threshold, radius, ramp time comes from TOML or is derived from mesh / parameters (`feedback_no_hardcoded_numbers`).
- **Audit ALL occurrences before removing a pattern** — Phase N removes `NucleationKind::StrengthReduction` and `NucleationKind::Overstress`; the implementer MUST grep every `NucleationKind::`, `OverstressSpec`, `OverstressPerDOFParams`, `ResolveOverstress`, `ResolveForcedRupture`, and `LSW_ForcedRupture` site in the spatial code path AND in `test_spatial_friction_resolver.cpp` AND in `tests/unit/test_phaseh_lsw_forced_rupture.cp` BEFORE the removal commit (`feedback_complete_sign_sites`).

Numerical constraints

- **CFL** — scalar `cp = √((λ + 2μ)/ρ)` from `[material_constant_fallback]`; `dt = cfl · h_min / cp`. Uniform under constant material.
- **smoothStep** — `smoothStep(t, t0) = 0` for `t ≤ 0`, `exp(τ²/(t·(t − 2·t0)))` for `0 < t < t0` (where `τ = t − t0`), 1 for `t ≥ t0`. C∞ everywhere except `t = 0`. Per-sub-step increment `ΔS(t, Δt, t0) = smoothStep(t, t0) − smoothStep(t − Δt, t0)` telescopes exactly over `[0, T_nuc]`.
- **Gaussian spatial factor** — `F(r) = exp(− ((dx / radius_dip_m)² + (ds / radius_strike_m)²))` where `(dx, ds)` are the DOF-to-centre offsets resolved in the fault-local `(dip, strike)` directions via each DOF's

FaultBasis. F(0) = 1; numerically zero outside ~ 3 · radius_*.

- **Accumulator zero-bound clamp** — dS <= 0.0 short-circuits to no-op (mirrors tpv104_nucleation.hpp:124 R3-004: guards against sub-ULP negative drift from subtracting two values close to 1.0).
- **dt finiteness guard** — MFEM_ASSERT(std::isfinite(dt) && dt > 0) at the top of any smoothStep increment call (mirrors tpv104_nucleation.hpp:64 R3-007).

Phase N: Gradual_overstress nucleation implementation

Goal

After Phase N, the spatial driver supports **exactly one** nucleation kind NucleationKind::GradualOverstress. The per-DOF accumulator Δτ · F(r) is built once at init, then injected per ADER sub-step into DOFData[i].tau1_nuc / tau2_nuc via a callback hook in Tpv205SubStepIterator. The obsolete StrengthReduction / Overstress enum values, their resolvers, and their dispatch into LSW_ForcedRupture are removed from the spatial code path. Native TPV* drivers are untouched.

Files to Create

- miniapps/seas/dynamic/spatial_nucleation.hpp — public API: structs, resolver, accumulator (≈ 200 LOC).
- miniapps/seas/dynamic/spatial_nucleation.cpp — implementation (≈ 250 LOC).
- miniapps/seas/tests/unit/test_spatial_nucleation.cpp — 12 unit tests (≈ 400 LOC).

Files to Modify

- miniapps/seas/spatial/code/spatial_friction.hpp — replace enum NucleationKind, struct OverstressSpec, struct OverstressPerDOFParams, add struct GradualOverstressSpec, extend NucleationSpec. Remove ForcedRupturePerDOFParams from the public API (kept in cpp internal namespace only if any helper still uses it; otherwise delete).
- miniapps/seas/spatial/code/spatial_friction.cpp — replace [nucleation] parser block (L700–L750) with gradual_overstress variant; remove ResolveOverstress body and ResolveForcedRupture body (delete declarations from the resolver class header too if no native TPV driver consumes them — verify with grep -rn 'ResolveForcedRupture' miniapps/seas/).
- miniapps/seas/dynamic/tpv205_substep_iterator.hpp — add ONE additive overload of AdvanceWithSubStepStates(...) that takes a std::function<void(real_t, real_t)> per-sub-step callback. Do NOT modify the existing signature (any existing tests must still link).
- miniapps/seas/dynamic/tpv205_substep_iterator.cpp — implement the new overload as a thin wrapper that injects the callback between sub-steps.
- miniapps/seas/drivers/spatial_dyn_driver.cpp — at
  - L797–L824: simplify the dispatch — always wave.SetFaultFrictionLaw(FaultFrictionLaw::LSW). Drop use_strength_reduction variable.
  - L941–L964: remove the ResolveForcedRupture / ResolveOverstress calls; replace with ResolveGradualOverstress.
  - L977–L989: pass empty T_forced_s / t0_decay_s to InitializeFaultDOFs_Spatial (the LSW path consumes only LSW params when the friction-law is LSW not LSW_ForcedRupture), or remove those args from the call entirely if the signature changes (see below).
  - L1193–L1208: replace the iterator SetSubSteps block with the new callback-aware call into AdvanceWithSubStepStates. Remove the deferred SetForcedRuptureMode comment.
  - L1276: pass the per-sub-step accumulator callback into AdvanceADERWithSubStep_Spatial (or wire it inside the wrapper — see Phase N Detailed Req. 6).
- miniapps/seas/dynamic/spatial_setup.hpp — **NO CHANGE.** The existing signature requires T_forced_s / t0_decay_s (validated at spatial_setup.hpp:194–200). R-003 fix: the driver supplies dummy vectors T_forced_s = 1.0e9, t0_decay_s = 0.0 (the existing "never forced" sentinel). These are inert under the LSW

dispatch (`FaultFrictionLaw::LSW`, not `LSW_ForcedRupture`) that Phase N enforces — the `EvaluateADER_LSW` kernel never consults them. No new overload, no API surface bloat. Driver wiring shown in Detailed Req. #7 below.

- `miniapps/seas/Makefile` — add `SPATIAL_NUCLEATION_SRC = dynamic/spatial_nucleation.cpp`
  - object rule + link to `seas_spatial_dyn_driver` and the new unit-test target. Mirror the existing `SPATIAL_FRICTION_*` pattern.
- `miniapps/seas/tests/unit/test_spatial_friction_resolver.cpp` — remove any test that asserts on `ResolveOverstress` / `ResolveForcedRupture` / `NucleationKind::Overstress` / `NucleationKind::StrengthReduction` (F-4, F-5, T-22 per `spatial_dynamic_rupture_fix_round7_2026-05-18.md:117-203`). Keep all other tests in this file unchanged.
- `miniapps/seas/safs/project_7.0_alternative/friction/EXAMPLE_spatial_friction_slip_weakening_safs.toml` — **already updated** to the new schema (no change needed in this phase; re-validate the file parses).

Files to Remove

None. All deletions are inside existing files (enum values, struct definitions, resolver bodies, test functions).

Interfaces

```
// dynamic/spatial_nucleation.hpp

namespace mfem { namespace seas { namespace spatial {

/// SCEC "Gaussian" smoothStep ramp (mirror of SmoothStep_TPV104, renamed
/// generic so the spatial driver does not depend on the TPV104 helper).
real_t SmoothStep(real_t current_time, real_t t0);

/// Per-sub-step smoothStep increment.  Asserts dt finite and positive.
real_t SmoothStepIncrement(real_t current_time, real_t dt, real_t t0);

/// Gaussian-shaped spatial factor F(r) in the fault-local (dip, strike) frame.
///
/// @param[in] dof_xyz       Physical coord (3 components) at the DOF.
/// @param[in] dof_basis_row One row of the 3*N x 3 DenseMatrix (basis vectors
///                          n, t1=dip, t2=strike per DOF).
/// @param[in] center_xyz    Hypocenter physical coords (3).
/// @param[in] radius_dip_m  e-fold radius along the dip direction (m, > 0).
/// @param[in] radius_strike_m  e-fold radius along strike (m, > 0).
/// @returns F(r) in [0, 1].  At dof == center, F = 1.  At dof outside
///          ~3 · max(radius_dip_m, radius_strike_m), F < 1e-4.
real_t GaussianFactorFaceLocal(const real_t* dof_xyz,
                               const real_t* dof_basis_dip,    // 3-vector
                               const real_t* dof_basis_strike, // 3-vector
                               const real_t* center_xyz,
                               real_t radius_dip_m,
                               real_t radius_strike_m);

struct GradualOverstressSpec
{
   real_t center_x_m          = 0.0;
   real_t center_y_m          = 0.0;
   real_t center_z_m          = 0.0;
   real_t radius_dip_m        = 0.0;   // > 0 required when enabled
   real_t radius_strike_m     = 0.0;   // > 0 required when enabled
```

```
   real_t delta_tau_dip_pa    = 0.0;   // may be 0
   real_t delta_tau_strike_pa = 0.0;   // may be 0
   real_t T_nuc_s             = 0.0;   // > 0 required when enabled (smoothStep t0)
   // R-007 fix: t0_smooth_s field DROPPED — redundant with T_nuc_s.
};

/// Per-DOF resolved gradual-overstress targets.
///   amplitude_dip(i)    = F(r_i) · delta_tau_dip_pa
///   amplitude_strike(i) = F(r_i) · delta_tau_strike_pa
///   radial(i)           = F(r_i)
/// All three vectors are empty when nucleation is disabled.
struct GradualOverstressPerDOFParams
{
   Vector amplitude_dip;
   Vector amplitude_strike;
   Vector radial;
};

/// Build per-DOF amplitudes from the spec + per-DOF coords + per-DOF basis.
/// `dof_coords_3d.Size() == 3 * N`; `dof_basis.Height() == 9`,
/// `dof_basis.Width() == N`.  Per DOF i (matching driver L228, L257,
/// L275-277): `dof_basis(0..2, i)` = normal, `dof_basis(3..5, i)` =
/// tangent1 = dip, `dof_basis(6..8, i)` = tangent2 = strike.  MFEM
/// DenseMatrix is column-major, so `&dof_basis(0, i)`, `&dof_basis(3, i)`,
/// `&dof_basis(6, i)` are contiguous 3-vector pointers usable by
/// `GaussianFactorFaceLocal` without copies (R-001 fix).
GradualOverstressPerDOFParams ResolveGradualOverstress(
   const GradualOverstressSpec& spec,
   bool                         enabled,
   const Vector&                dof_coords_3d,
   const DenseMatrix&           dof_basis);

/// Apply one ADER sub-step's worth of gradual_overstress increment to the
/// per-DOF DOFData::tau1_nuc / tau2_nuc fields.  No-op (early-return)
/// when `t_substep_end >= T_nuc_s` (the smoothStep is already at 1) or
/// when `dS <= 0.0` (round-off guard).
///
/// Pattern (mirrors tpv104_nucleation.hpp ApplyNucleationIncremental_TPV104,
/// generalized to write both tau{1,2}_nuc):
///   Δ = SmoothStepIncrement(t_substep_end, dt_substep, t0_smooth_s)
///   for every QP i:
///      tau1_nuc[i] += Δ · amplitude_dip(i)
///      tau2_nuc[i] += Δ · amplitude_strike(i)
///   sigma_n_nuc is left at 0 (this mechanism does not perturb σ_n).
void ApplyGradualOverstressIncrement(
   std::vector<DOFData>&                dof_data,
   const GradualOverstressPerDOFParams&  params,
   real_t                               T_nuc_s,         // smoothStep t0
   real_t                               t_substep_end,
   real_t                               dt_substep);
// R-007 fix: drop the redundant `t0_smooth_s` parameter — the
// smoothStep ramp duration is `T_nuc_s` (matches TPV104 convention
// where TPV104Params::nuc_T plays both roles).  The schema's
// `t0_smooth_s` field is also dropped (see schema-doc edit).
```

```
}}}  // namespace

// spatial/code/spatial_friction.hpp (REPLACE existing NucleationKind /
// OverstressSpec / OverstressPerDOFParams / ForcedRupturePerDOFParams)

namespace mfem { namespace seas { namespace spatial {

enum class NucleationKind { GradualOverstress };

struct NucleationSpec
{
   NucleationKind          kind     = NucleationKind::GradualOverstress;
   bool                    enabled  = false;
   GradualOverstressSpec   gradual_overstress;   // populated when enabled
};

}}}  // namespace

// dynamic/tpv205_substep_iterator.hpp  (PURELY ADDITIVE OVERLOAD)

class Tpv205SubStepIterator
{
public:
   // ... existing API unchanged ...

   /// @brief Callback-aware variant of AdvanceWithSubStepStates.
   ///
   /// The callback is invoked ONCE per ADER sub-step BEFORE the per-QP
   /// friction pipeline (mirrors Tpv104SubStepIterator::Advance line 295's
   /// internal call to ApplyNucleationIncremental_TPV104).
   ///
   /// `nuc_callback(t_substep_end, dt_substep)` writes into the same
   /// `dof_data` reference passed below — it MUST mutate only
   /// `DOFData::tau{1,2}_nuc / sigma_n_nuc` channels.  Callers may use
   /// `[](real_t, real_t){}` to opt out (no-op nucleation).
   void AdvanceWithSubStepStates(
      std::vector<DOFData>&                                  dof_data,
      const std::vector<Vector>&                             fault_coords,
      const std::vector<std::vector<real_t>>&                Q_pointwise_plus_per_substep,
      const std::vector<std::vector<real_t>>&                Q_pointwise_minus_per_substep,
      real_t                                                 dt_macro,
      real_t                                                 t_macro_start,
      real_t*                                                I_imp_plus_flat,
      real_t*                                                I_imp_minus_flat,
      const std::function<void(real_t, real_t)>&             nuc_callback);
};
```

Detailed Requirements

1. **Replace NucleationKind enum** (one greppable change). Verify every call site compiles by running:

   ```
   grep -rn 'NucleationKind::' miniapps/seas/
   ```

   Every match outside `dynamic/tpv104_nucleation.hpp` and the native TPV driver code (TPV104 may still use `LSW_ForcedRupture` in its own workflow — verify it does NOT reference `NucleationKind`) must be `Nucle-`

ationKind::GradualOverstress. The native TPV104 driver does NOT reference NucleationKind (it has its own enum/state); confirm with grep -n 'NucleationKind' miniapps/seas/drivers/tpv104_driver.cpp returns empty.

2. **SmoothStep / SmoothStepIncrement bodies** — copy from tpv104_nucleation.hpp:40–69 verbatim, removing the _TPV104 suffix and the TPV104Params::nuc_T reference. The dt > 0 finiteness assert stays. Add unit tests covering: t=0 returns 0, t=t0 returns 1, t=2·t0 returns 1, t<0 returns 0, smoothStep continuity at t=0 and t=t0 (left and right limits agree to machine precision), increment positivity over [0, t0], increment exact telescope to 1 when summed over a partition of [0, t0] with arbitrary dt.

3. **GaussianFactorFaceLocal body** — projects the (dof_xyz − center_xyz) vector onto the fault-local dip and strike directions, then evaluates exp(− ((dx / radius_dip)² + (ds / radius_strike)²)). Asserts both radii are > 0. Returns 0.0 immediately if the exponent overflows below −700 (numerical floor) to avoid exp(−large) = 0 denormal underflow noise. Unit tests: F(centre) == 1.0 (to machine precision); F(radius_dip * sqrt(ln(2)), 0) == 0.5 (the half-amplitude radius); F outside 3·max(radius_*) < 1e-4; rotation invariance under arbitrary fault basis.

4. **ResolveGradualOverstress body** — iterate the N DOFs; for each DOF i, pass &dof_basis(3, i) (dip, contiguous 3-vector) and &dof_basis(6, i) (strike, contiguous 3-vector) to GaussianFactorFaceLocal. The layout is (9, N) column-major per drivers/spatial_dyn_driver.cpp:228, 257, 275–277: rows 0..2 = normal, rows 3..5 = dip, rows 6..8 = strike. Then write amplitude_dip(i) = F(r_i) · spec.delta_tau_dip_pa amplitude_strike(i) = F(r_i) · spec.delta_tau_strike_pa radial(i) = F(r_i). When !enabled, return three zero-sized Vectors. Unit test: a 5-DOF fixture where the centre is at DOF 2, expect radial(2) == 1.0, radial(0) == radial(4) (symmetry), amplitude_∗ matches the spec δτ at the centre. (R-001 fix: previous layout claim (3∗N, 3) was wrong; actual layout is (9, N) column-major.)

5. **ApplyGradualOverstressIncrement body** — mirror the pattern of ApplyNucleationIncremental_TPV104 exactly, except:
   - write BOTH tau1_nuc and tau2_nuc (TPV104 wrote only tau2_nuc because TPV104 is pure strike-slip; SAFS curvilinear fault has variable strike, so the per-DOF amplitudes are already projected into the local (dip, strike) directions);
   - sigma_n_nuc is left at 0 (this mechanism does NOT perturb σ_n);
   - early-return when params.amplitude_dip.Size() == 0 (nucleation disabled);
   - early-return when t_substep_end >= T_nuc_s + dt_substep (the accumulator has already telescoped to its full target; not even a sub-ULP increment remains).

6. **Per-sub-step hook in Tpv205SubStepIterator** — add ONE new method (overload) AdvanceWithSubStep-States(..., const std::function<void(real_t, real_t)>& nuc_callback). Implementation: copy the existing AdvanceWithSubStepStates body verbatim, insert the callback invocation at the top of each iteration of the for (o = 0; o < 0; ++o) sub-step loop, BEFORE the per-QP friction pipeline (mirrors tpv104_substep_iterator.cpp:295). The callback receives (t_substep_end, dt_sub). The original (no-callback) overload is kept for backward compatibility and routes through the new overload with a no-op [](real_t, real_t){} callback.

7. **Driver-side wiring** at spatial_dyn_driver.cpp (use line numbers as anchors; they will shift):

```
// After step 12 (replaces ResolveForcedRupture / ResolveOverstress):
spatial::GradualOverstressPerDOFParams nuc_params =
   spatial::ResolveGradualOverstress(
      cfg.nucleation.gradual_overstress,
      cfg.nucleation.enabled,
      dof_coords_3d,
      dof_basis);

// R-003 fix: InitializeFaultDOFs_Spatial requires T_forced_s and
// t0_decay_s vectors of size num_fault_total (validated at
// spatial_setup.hpp:194-200).  Under the LSW dispatch (not
```

```
// LSW_ForcedRupture) these are NEVER read — supply the "never
// forced" sentinel so the size validator passes:
Vector dummy_T_forced(num_fault_total);  dummy_T_forced = 1.0e9;
Vector dummy_t0_decay(num_fault_total);  dummy_t0_decay = 0.0;

// Step 14 (InitializeFaultDOFs_Spatial) — pass the dummy vectors
// in place of the deleted fr.T_forced_s / fr.t0_decay_s:
spatial::InitializeFaultDOFs_Spatial<ParMesh>(
   dof_data, num_fault_total, dof_to_elem, material, pmesh,
   lsw, geom.GetTauPre(), geom.sigma_n_per_dof(),
   dummy_T_forced, dummy_t0_decay,
   dof_ips);

// Inside step 19 (Tpv205SubStepIterator setup), the per-sub-step hook:
auto nuc_cb = [&nuc_params, &dof_data, &cfg]
              (real_t t_sub_end, real_t dt_sub)
{
   if (!cfg.nucleation.enabled) { return; }
   spatial::ApplyGradualOverstressIncrement(
      dof_data, nuc_params,
      cfg.nucleation.gradual_overstress.T_nuc_s,
      t_sub_end, dt_sub);     // R-007 fix: pass only T_nuc_s
};

// Pass nuc_cb into AdvanceADERWithSubStep_Spatial (extend that
// wrapper's signature to take and forward the callback).
```

8. **`AdvanceADERWithSubStep_Spatial` signature extension** — the wrapper at `spatial_dyn_driver.cpp:334` gains an additional trailing arg `const std::function<void(real_t, real_t)>& nuc_callback`. Inside, the call to `iterator.AdvanceWithSubStepStates(...)` is upgraded to the new callback overload added in Phase N Detailed Req. 6.

9. **TOML parser update in `spatial_friction.cpp`** — replace the `[nucleation]` block parser (`L700–L750`) with:

```
if (root.contains("nucleation"))
{
   const auto& nuc = root.at("nucleation");
   cfg.nucleation.enabled = true;

   const std::string kind_s = toml_str(nuc, "kind", "");
   MFEM_VERIFY(kind_s == "gradual_overstress",
               "[nucleation].kind must be \"gradual_overstress\" "
               "(the only supported kind in this driver); got '"
               << kind_s << "'");
   cfg.nucleation.kind = NucleationKind::GradualOverstress;

   MFEM_VERIFY(nuc.contains("gradual_overstress"),
               "[nucleation] kind=\"gradual_overstress\" requires a "
               "[nucleation.gradual_overstress] sub-block");
   const auto& g = nuc.at("gradual_overstress");
   auto& gs = cfg.nucleation.gradual_overstress;
   gs.center_x_m          = toml_real(g, "center_x_m",          0.0);
   gs.center_y_m          = toml_real(g, "center_y_m",          0.0);
   gs.center_z_m          = toml_real(g, "center_z_m",          0.0);
   gs.radius_dip_m        = toml_real(g, "radius_dip_m",        0.0);
```

```
   gs.radius_strike_m     = toml_real(g, "radius_strike_m",     0.0);
   gs.delta_tau_dip_pa    = toml_real(g, "delta_tau_dip_pa",    0.0);
   gs.delta_tau_strike_pa = toml_real(g, "delta_tau_strike_pa", 0.0);
   gs.T_nuc_s             = toml_time_seconds(g, "T_nuc_s",     0.0);
   // R-007 fix: t0_smooth_s DROPPED — T_nuc_s plays both roles.

   MFEM_VERIFY(gs.radius_dip_m    > 0.0,
               "[nucleation.gradual_overstress].radius_dip_m must be > 0; "
               "got " << gs.radius_dip_m);
   MFEM_VERIFY(gs.radius_strike_m > 0.0,
               "[nucleation.gradual_overstress].radius_strike_m must be > 0; "
               "got " << gs.radius_strike_m);
   MFEM_VERIFY(gs.T_nuc_s         > 0.0,
               "[nucleation.gradual_overstress].T_nuc_s must be > 0; "
               "got " << gs.T_nuc_s);
}
// else: enabled stays false; driver runs without nucleation perturbation.
```

10. **Remove dead code, in this order, in ONE atomic commit** (the TPV/BP5 byte-exact regression gate is the safety net):
    - From `spatial_friction.hpp`: delete `OverstressSpec`, `OverstressPerDOFParams`, `ForcedRupturePerDOFParams`, declarations of `SpatialFrictionResolver::ResolveOverstress` and `SpatialFrictionResolver::ResolveForcedRupture`. Old `NucleationKind::StrengthReduction` / `Overstress` enum values are replaced (item 1).
    - From `spatial_friction.cpp`: delete `resolve_forced_impl` template and both `ResolveForcedRupture` non-template wrappers (`L1087–L1196`); delete `ResolveOverstress` body (`L1215–L1243`); delete the `nuc.contains("overstress")` sub-block in the parser (will be replaced by the gradual_overstress parser in item 9).
    - From `spatial_dyn_driver.cpp`: delete the `use_strength_reduction` variable and the conditional `SetFaultFrictionLaw(...)` dispatch (always `LSW`); delete the `ResolveForcedRupture` / `ResolveOverstress` calls; delete the `[[maybe_unused]] OverstressPerDOFParams overstress` line.
    - From `test_spatial_friction_resolver.cpp`: delete F-4 (`F_4_overstress_kind_returns_sentinel`), F-5 (`F_5_resolve_overstress_stub_behaviour`), T-22 (`T_22_nucleation_kind_overstress_parses`), and any other test that references `NucleationKind::StrengthReduction` / `Overstress`, `ResolveOverstress`, `ResolveForcedRupture`, `OverstressSpec`, or `ForcedRupturePerDOFParams`. Run `grep -rn 'OverstressSpec\| OverstressPerDOFParams\|ForcedRupturePerDOFParams\|ResolveOverstress\|ResolveForcedRupture\|StrengthReduction' miniapps/seas/tests/` after removal to verify zero remaining references in test code.

11. **Native TPV* / BP5 isolation audit** (the safety gate): before committing Phase N, run

```
grep -rn 'NucleationKind\|GradualOverstress\|spatial_nucleation' \
   miniapps/seas/dynamic/tpv102_setup_total.hpp \
   miniapps/seas/dynamic/tpv104_setup.hpp \
   miniapps/seas/dynamic/tpv104_nucleation.hpp \
   miniapps/seas/dynamic/tpv205_setup.hpp \
   miniapps/seas/drivers/tpv102_driver.cpp \
   miniapps/seas/drivers/tpv104_driver.cpp \
   miniapps/seas/drivers/tpv205_driver.cpp \
   miniapps/seas/bp5/ miniapps/seas/bp1/ miniapps/seas/bp2/
```

    Output MUST be empty. Otherwise the change is leaking into the byte- exact regression contract.

Edge Cases to Handle

- **`[nucleation]` block absent** — `cfg.nucleation.enabled == false`; resolver returns empty Vectors; the per-sub-step accumulator early-returns (no-op); driver runs with `tau1_nuc / tau2_nuc / sigma_n_nuc` all zero (initialised by `InitializeFaultDOFs_Spatial`).
- **`enabled == true` but `delta_tau_dip_pa == 0 && delta_tau_strike_pa == 0`** — `amplitude_dip(i) == amplitude_strike(i) == 0` everywhere; the per-sub-step accumulator runs but writes zeros. Not an error.
- **`t_substep_end >= T_nuc_s`** — `SmoothStepIncrement` returns 0; the accumulator early-returns at `dS <= 0`.
- **DOF with `dof_basis` containing a degenerate row** (zero-normal fallback; see `FaultGeometry::NumZeroNormalFallbacks`) — `R-001 TODO`: document that `GaussianFactorFaceLocal` will silently produce a non-zero but rotated-incorrectly value at such a DOF. The driver's existing `--print-derived` (Phase D) reports `geom.NumZeroNormalFallbacks()`; if non-zero, log a rank-0 warning.
- **`Tpv205SubStepIterator` original (no-callback) overload called from tests** — verify by grep; if any unit test calls the old overload it must continue to work (the no-callback path is preserved as a `nuc_callback == no-op` wrapper).
- **Multi-rank**: each rank owns disjoint fault DOFs; `nuc_callback` writes only into LOCAL `dof_data` entries. No MPI exchange needed.
- **Restart from V1 checkpoint**: the V1 checkpoint stores `dof_data` (which includes `tau1_nuc / tau2_nuc`). On restart at `t_restart > 0`, the per-sub-step accumulator computes `ΔS(t_restart_end, dt_sub, t0)` which is correct because smoothStep depends only on absolute time, not on cumulative-so-far state. No special restart logic needed.

Acceptance Criteria

- ☐ **T-N01 — SmoothStep boundary values**: `SmoothStep(0, 1) == 0`, `SmoothStep(1, 1) == 1`, `SmoothStep(2, 1) == 1`, `SmoothStep(-0.5, 1) == 0`. Greppable as `T_N01_smoothstep_boundary`.
- ☐ **T-N02 — SmoothStepIncrement telescope**: for `t0 = 1.0`, a uniform partition of `[0, 1]` into 100 sub-steps `dt = 0.01` must sum the increments to `SmoothStep(1, 1) - SmoothStep(0, 1) == 1.0 ± 1e-12`. Tag `T_N02_smoothstep_telescope_uniform`.
- ☐ **T-N03 — SmoothStepIncrement non-uniform telescope**: random positive partition of `[0, 1]` summing to 1 also telescopes to 1.0 ± 1e-12. Tag `T_N03_smoothstep_telescope_nonuniform`.
- ☐ **T-N04 — SmoothStepIncrement dt assert**: `SmoothStepIncrement(0.5, 0.0, 1.0)` aborts (MFEM_ASSERT); `SmoothStepIncrement(0.5, NaN, 1.0)` aborts.
- ☐ **T-N05 — GaussianFactor centre value**: at `dof_xyz == centre`, `F == 1.0 ± 1e-15`.
- ☐ **T-N06 — GaussianFactor radial decay**: at radial distance `r = radius_dip · sqrt(ln(2))` along pure dip direction, `F == 0.5 ± 1e-12`.
- ☐ **T-N07 — GaussianFactor anisotropy**: with `radius_dip = 2 · radius_strike`, off-axis decay is correctly anisotropic.
- ☐ **T-N08 — ResolveGradualOverstress disabled returns empty**: when `enabled == false`, all three output Vectors have `Size() == 0`.
- ☐ **T-N09 — ResolveGradualOverstress 5-DOF fixture**: 5 DOFs spaced along a single fault line, centre at DOF 2; `radial(2) == 1.0`, `amplitude_strike(2) == spec.delta_tau_strike_pa`, symmetry `radial(0) == radial(4)`.
- ☐ **T-N10 — ApplyGradualOverstressIncrement telescopes per DOF**: 100 sub-steps over `[0, T_nuc]`, after the loop `dof_data[i].tau2_nuc == params.amplitude_strike(i) ± 1e-10`.
- ☐ **T-N11 — ApplyGradualOverstressIncrement post-T_nuc no-op**: at `t = 2 · T_nuc`, the increment is exactly 0 and `tau{1,2}_nuc` is unchanged.
- ☐ **T-N12 — Tpv205SubStepIterator callback overload contract**: construct an iterator with O=4 sub-steps, pass a callback that records `(t_sub_end, dt_sub)` pairs; assert the callback was invoked exactly 4 times with the expected `t_sub_end` values.
- ☐ **TPV/BP5 byte-exact regression stays green** — `make test-tpv104`, `make test-tpv205`, `make test-bp5-smoke`, `make test-bp5-integration`, `make test-tpv102-local` (where applicable to this branch) all bit-identical to pre-Phase-N baseline.
- ☐ **`seas_spatial_dyn_driver` build** — `make seas_spatial_dyn_driver` succeeds; `make test-spatial-dyn-driver` (dry-run smoke) succeeds.

Dependencies

- Depends on: nothing (all new code is additive in `dynamic/`; the destructive deletions are inside the spatial code path only).
- Required by: Phase D (the print-derived gate consults `nuc_params` to report `F(r=0) · |Δτ|` vs `(μ_s − μ_d) · σ_n_eff`); Parity Phase 6 (the `nuc_amplitude` / `nuc_radial_factor` ParaView static fields are populated from `nuc_params`).

Phase D: Real `--print-derived` implementation + initial-equilibrium gate

Goal

Replace the 3-line `--print-derived` stub at `spatial_dyn_driver.cpp:1023-1027` with a full pre-flight pass that prints physical histograms and **aborts** when the initial conditions are ill-posed. A user that runs `--dry-run --print-derived` on Frontera login gets a hard fail before submitting if their stress / friction / nucleation setup would spontaneously rupture or fail to nucleate.

Files to Create

- `miniapps/seas/dynamic/spatial_print_derived.hpp` — single-header API for the pre-flight printer (≈ 60 LOC interface). Implementation in same header or in a sibling `.cpp` if cross-TU usage is desired.
- `miniapps/seas/dynamic/spatial_print_derived.cpp` — implementation (≈ 200 LOC).

Files to Modify

- `miniapps/seas/drivers/spatial_dyn_driver.cpp` — replace L1023-L1027 with a call to `spatial::PrintDerivedAndCheck(...)`. Insert AFTER the LSW resolver call (L939) and AFTER the `ResolveGradualOverstress` call (added in Phase N), AND after the per-DOF stress assignment that produces `geom.GetTauPre()` / `geom.sigma_n_per_dof()` (L924+). The call signature takes everything it needs from those already-computed sources.
- `miniapps/seas/Makefile` — add `SPATIAL_PRINT_DERIVED_OBJ` and link.

Interfaces

```
// dynamic/spatial_print_derived.hpp

namespace mfem { namespace seas { namespace spatial {

struct PrintDerivedConfig
{
   bool   enabled               = false;  // mirrors --print-derived flag
   bool   abort_on_failure      = true;   // false ⇒ warn-only
   real_t outside_safety_factor = 3.0;    // r > F · max(radius_*) ⇒ "outside"
};

/// Pre-flight pass.  Prints per-region histograms + L_nuc + max-ratio
/// statistics on rank 0; aborts (MFEM_ABORT) on initial-equilibrium
/// violation when `cfg.abort_on_failure == true`.
///
/// @returns the maximum |τ_pre| / (μ_s · σ_n_eff) ratio observed at any
///          DOF OUTSIDE the nucleation Gaussian's support
///          (`r > outside_safety_factor · max(radius_dip, radius_strike)`).
///          When `cfg.enabled == false`, no I/O; returns 0.0 and is a
///          no-op.
```

```
real_t PrintDerivedAndCheck(
   const PrintDerivedConfig&               cfg,
   const SlipWeakeningPerDOFParams&        lsw,
   const Vector&                           tau_pre_per_dof,      // 2 * N (interleaved [dip, strike])
   const Vector&                           sigma_n_eff_per_dof,  // N
   const Vector&                           dof_coords_3d,        // 3 * N
   const NucleationSpec&                   nuc,
   const GradualOverstressPerDOFParams&    nuc_params,
   const StressSpec&                       stress,               // for σ_1 azimuth
   real_t                                  dt_cfl,
   int                                     num_zero_normal_fallbacks,
   MPI_Comm                                comm,
   int                                     rank,
   std::ostream&                           out = std::cout);

}}}  // namespace
```

Detailed Requirements

1. **Per-DOF `L_nuc` computation and histogram**. For each DOF i: `L_nuc(i) = μ_bulk · d_c(i) / ((μ_s(i) − μ_d(i)) · σ_n_eff(i))` where `μ_bulk` is the scalar `material.mu_const` (constant material). Skip barrier DOFs (`μ_s(i) >= 0.5 · 1.0e6`, the sentinel). Compute min, max, mean, median across all NON-barrier DOFs (use `MPI_Allreduce` for min/max; gather to rank 0 for median). Print as a 5-bin histogram in metres. Also report `L_nuc / h_min` ratio (should be ≥ 10 for the nucleation patch).

2. **`|τ_pre| / (μ_s · σ_n_eff)` outside-asperity ratio.** For each DOF:

   ```
   r_i = || dof_coords_3d(i) − nuc.center ||  (Euclidean, not fault-local)
   r_threshold = cfg.outside_safety_factor · max(radius_dip, radius_strike)
   if (r_i > r_threshold OR !nuc.enabled):
      ratio = sqrt(tau_pre(2i)² + tau_pre(2i+1)²) / (μ_s(i) · σ_n_eff(i))
      max_outside_ratio = max(max_outside_ratio, ratio)
      // skip barrier DOFs
   ```

   After the local pass, `MPI_Allreduce(MPI_MAX)` to all ranks. If `max_outside_ratio >= 1.0` AND `cfg.abort_on_failure`, MFEM_ABORT with the offending ratio value AND the worst DOF's (x, y, z). Otherwise, emit a `[derived]` rank-0 WARNING line if ratio □ [0.9, 1.0) (close to the failure threshold).

3. **Hypocenter-patch nucleation budget check.** When `nuc.enabled`: for each DOF inside `r_i < r_threshold`, compute the local nucleation budget `budget(i) = (μ_s(i) − μ_d(i)) · σ_n_eff(i)` and the per-DOF gradual_overstress amplitude `A(i) = sqrt(nuc_params.amplitude_dip(i)² + nuc_params.amplitude_strike(i)²)`. Find `i* = argmax(A(i) − budget(i))` (most-overstressed DOF). If `A(i*) < budget(i*) · 0.9` (the patch barely nucleates) AND `cfg.abort_on_failure`, MFEM_ABORT with the gap and DOF coords. The 90% threshold gives some headroom — exact 100% may not nucleate due to slip-weakening Brent solver chatter.

4. **σ_1 azimuth + max-shear direction** for the input stress tensor (R-004 fix: cite the exact MFEM API and atan2 ordering). For `kind = constant_tensor`, build the symmetric 3×3 from the six `[stress].sigma_*_pa` keys and call MFEM's eigensolver:

   ```
   mfem::DenseMatrix S(3, 3);
   S(0,0) = sigma_xx; S(1,1) = sigma_yy; S(2,2) = sigma_zz;
   S(0,1) = S(1,0) = sigma_xy;
   S(1,2) = S(2,1) = sigma_yz;
   S(0,2) = S(2,0) = sigma_xz;
   mfem::Vector eigvals(3);
   mfem::DenseMatrix eigvecs(3, 3);
   ```

```
S.Eigensystem(eigvals, eigvecs);    // MFEM returns ascending order
const int i1 = 2;                   // σ_1 = LARGEST (most compressive)
const real_t v1x = eigvecs(0, i1);
const real_t v1y = eigvecs(1, i1);
const real_t v1z = eigvecs(2, i1);
```

Azimuth (radians clockwise from north in EAST-NORTH frame; x = E, y = N) — **note the (x, y) atan2 ordering, NOT (y, x)**:

```
const real_t azimuth_rad = std::atan2(v1x, v1y);    // (x, y) order
real_t azimuth_deg = azimuth_rad * 180.0 / M_PI;
if (azimuth_deg < 0.0) { azimuth_deg += 360.0; }    // wrap to [0, 360)
```

Plunge (degrees from horizontal; positive = down):

```
const real_t v1h = std::sqrt(v1x * v1x + v1y * v1y);
const real_t plunge_deg = std::atan2(-v1z, v1h) * 180.0 / M_PI;
```

For `kind = sidecar_hdf5`, report the per-fault-DOF max `|τ_pre|` and the depth at which it occurs; azimuth computation is non-trivial in the sidecar case (depth-dependent eigenframe), so emit `"[derived] σ_1 azimuth from sidecar: TBD (depth-dependent; see stress sidecar README)"` instead.

5. **CFL Δt min/max** — the spatial driver under constant material has a uniform `dt_cfl` (already computed at L1002 as `wave.ComputeMaxDt(cfl)`). Report `dt_cfl`, total steps `nsteps = ceil(tfinal / dt_cfl)`, and the per-DOF heterogeneous CFL min/max (which will equal `dt_cfl` under constant material; this prepares the print for the future heterogeneous- material case).

6. **LSW parameter histograms** — for `μ_s`, `μ_d`, `d_c`, compute per-region (or per-depth-band if `[[friction.slip_weakening.spatial]]` has no `region_attr` rule) min/max/mean and print as a 5-bin histogram.

7. **`NumZeroNormalFallbacks` reporting** — call `geom.NumZeroNormalFallbacks()` (already exposed by `FaultGeometry`). If > 0, emit a `[derived] WARNING: <N> fault DOFs hit the zero-normal fallback; gradual_overstress F(r) at those DOFs may be miscomputed. Seespatial_workflow_safs_runbook_2026-05-18.md§5 for remediation.`

8. **Output format** — every line prefixed with `[derived]`. Use a structured layout that's both human-readable AND grep-able by downstream Python tools:

```
[derived] num_fault_global = 250000
[derived] cp = 4877.5 m/s, cs = 3458.7 m/s
[derived] dt_cfl = 6.84e-05 s, nsteps = 175438
[derived] mesh h_min = 333.3 m (from --no-sidecar-material, scalar cp)
[derived] L_nuc min/max/mean = 1234.5 / 6789.0 / 3456.7 m
[derived] L_nuc / h_min min/max = 3.7 / 20.4 (NB: should be >= 10 in nucleation patch)
[derived] |tau_pre| / (mu_s * sigma_n_eff) max OUTSIDE asperity = 0.847
[derived] nucleation: enabled
[derived] nucleation peak |F(r) * delta_tau| at DOF (x=480000, y=3750000, z=-7000) = 2.50e+07 Pa
[derived] nucleation budget at the same DOF (mu_s - mu_d) * sigma_n_eff = 1.20e+07 Pa
[derived] nucleation overshoot = 13.0 MPa (sufficient)
[derived] sigma_1 azimuth = 47.3 deg, plunge = 12.5 deg, magnitude = 1.05e+08 Pa
[derived] fault DOFs with zero-normal fallback: 0
[derived] PASS: initial conditions are well-posed.
```

Edge Cases to Handle

- **No nucleation block** (`!nuc.enabled`) — skip items 3 (budget check is meaningless) AND treat every DOF as "outside the asperity" in item 2. Emit a `[derived] note: no [nucleation] block; the simulation will run without a nucleation perturbation. Set tau_pre such that some DOFs exceed the friction`

strength. If max_outside_ratio < 1.0 in this case, the run cannot nucleate at all; MFEM_ABORT with "no nucleation block AND max |τ_pre| / (μ_s · σ_n_eff) = X < 1.0 □ this configuration will not produce any rupture."
- **All DOFs are barriers** (corner case) — skip every per-DOF check, emit "[derived] all fault DOFs are barriers; nothing will rupture", abort.
- **abort_on_failure == false** (env var override, e.g. SEAS_SKIP_EQUILIBRIUM_GATE=1) — log every failure as a WARNING but do not abort. The driver still exits 0 after the dry-run. Used for parameter-sweep tooling that knows it's submitting unphysical setups to map the boundary.

Acceptance Criteria

- □ **T-D01 — Pre-flight passes on the EXAMPLE TOML**: seas_spatial_dyn_driver --config friction/EXAMPLE_spatial_friction_slip_weakening_safs.toml --no-sidecar-material --mesh ...zgraded.msh --dry-run --print-derived exits 0 with [derived] PASS on the last line. (Caveat: this depends on the EXAMPLE TOML being a well-posed setup — see implementation note Risk Assessment §2.)
- □ **T-D02 — Pre-flight ABORTS on supercritical setup**: a TOML with [stress] kind = "constant_tensor", sigma_xy_pa = 5.0e8 (500 MPa pure shear — physically unreasonable, will produce per-DOF |τ_pre| > 100 MPa everywhere) AND mu_s_default = 0.4, mu_d_default = 0.3 aborts at --dry-run with a max OUTSIDE asperity = X.XX message where the ratio exceeds 1. (R-002 fix: inflating delta_tau_strike_pa does NOT trigger this gate — that field controls the nucleation amplitude, which is the INSIDE-asperity check T-D03 below. The outside-asperity ratio uses tau_pre from the stress source.)
- □ **T-D03 — Pre-flight ABORTS on insufficient nucleation**: a TOML with delta_tau_strike_pa = 1.0e4 (way below (μ_s − μ_d) · σ_n_eff ≈ 1e7) aborts with nucleation overshoot = −X MPa (INSUFFICIENT).
- □ **T-D04 — Pre-flight WARNs on near-threshold**: a TOML with the outside-asperity ratio in [0.9, 1.0) prints WARNING: max OUTSIDE ratio = 0.95 and continues.
- □ **T-D05 — SEAS_SKIP_EQUILIBRIUM_GATE=1 honored**: same supercritical config as T-D02 with the env var set exits 0 (WARN only).
- □ **T-D06 — σ_1 azimuth correctness**: a [stress] kind = "constant_tensor" with σ_xx = σ_yy = 50e6, σ_xy = 25e6, others = 0 produces sigma_1 azimuth = 45 deg ± 0.1 deg. (Computed in the EAST-NORTH frame per the schema convention; pure shear at 45° gives σ_1 along NE.)
- □ **T-D07 — dt_cfl matches wave.ComputeMaxDt**: the pre-flight's dt_cfl value equals the value computed at spatial_dyn_driver.cpp:1002 exactly (this is sanity that the pre-flight isn't re-computing wrong).

Dependencies

- Depends on: Phase N (needs GradualOverstressPerDOFParams, NucleationSpec.gradual_overstress).
- Required by: Phase Z (the smoke sbatch invokes --dry-run --print-derived as its first step; the verifier asserts exit 0 on a known-good fixture TOML).

Parity Phases 1-6: Full ParaView paraview-compaction feature surface

Goal

Implement all six phases of safs/project_7.0_alternative/debug_document/spatial_paraview_compaction_parity_plan_2026-05-18.md as written, with the SAFS fault static fields using nuc_amplitude (= F(r) · sqrt(Δτ_dip² + Δτ_strike²)) and nuc_radial_factor (= F(r) ∈ [0, 1]) instead of the obsolete T_forced_rupture (already updated in the parity plan).

Files to Create / Modify

The parity plan is the **single source of truth** for files-to-modify and detailed requirements per parity sub-phase. This plan does NOT duplicate those requirements; it treats them as inputs. The implementer reads the parity plan section-by-section and implements Phase 1 → 2 → 3 → 4 → 5 → 6 in order. Each parity phase has its own acceptance criteria in the parity plan; they all roll up here as a single "Parity Phases 1-6 acceptance" line below.

Detailed Requirements

1. **Implement Parity Phase 1** (CLI flag surface + 6 build-flag guards). Reference: `spatial_paraview_compaction_parity_plan_2026-05-18.md` lines 102-211. Anchors in `spatial_dyn_driver.cpp` may have shifted by Phase N and Phase D; the implementer must re-locate the insertion points by searching for the existing CLI parsing block (`HasFlag(argc, argv, "--paraview-fault-zfp-tol")` rather than relying on the line numbers in the parity plan.
2. **Implement Parity Phase 2** (9 new `OutputSpec` fields + TOML parser + validator + schema doc; flip defaults `paraview_volume` / `paraview_bulk` from `"hdf5"` to `"off"`). Reference: parity plan lines 215-326. The schema doc edit lands in `safs/project_7.0_alternative/document/spatial_friction_config_schema.md` `[output]` table after `max_snapshots`; the default-flip footnote is mandatory. **Audit existing EXAMPLE TOML and the runbook for the default flip** — if either references `paraview_volume = "hdf5"` as the "default" anywhere in prose, update.
3. **Implement Parity Phase 3** (default-OFF master enable gate + rank-0 banner). Reference: parity plan lines 330-411. The banner pattern at `tpv104_driver.cpp:2117-2148` is the reference.
4. **Implement Parity Phase 4** (secondary `pv_bulk_out` collection: 6 stress components → `<output_dir>/ParaView_bulk/stress.`. Reference: parity plan lines 415-541. **Critical static_assert** (from parity plan Risk Assessment §2):

   ```
   static_assert(SXX == 0 && SYY == 1 && SZZ == 2
                 && SXY == 3 && SYZ == 4 && SXZ == 5,
                 "Phase 4 sigma memcpy assumes the wave_state.hpp ordering.");
   ```

5. **Implement Parity Phase 5** (primary collection `velocity` 3-comp L2 + `mpi_rank` L2 p=0 fields registered via `RegisterDomainField`). Reference: parity plan lines 545-629. **Critical static_assert**:

   ```
   static_assert(VX == 6 && VY == 7 && VZ == 8,
                 "Phase 5 velocity memcpy assumes contiguous VX/VY/VZ; "
                 "wave_state.hpp ordering changed.");
   ```

6. **Implement Parity Phase 6** (new `seas::ParaViewOutput::SetFaultParamsSpatial(...)` method publishing 8 per-DOF static arrays: `lsw_mu_s`, `lsw_mu_d`, `lsw_d_c`, `nuc_amplitude`, `nuc_radial_factor`, `sigma_n_init`, `tau1_init`, `tau2_init` + `SetTotalRunTime(tfinal)` for regime-adaptive cadence). Reference: parity plan lines 632-758. The per-DOF `nuc_amplitude` and `nuc_radial_factor` are computed by Phase N's `ResolveGradualOverstress`:

   ```
   pv_nuc_amplitude(i) = std::sqrt(
      nuc_params.amplitude_dip(i)    * nuc_params.amplitude_dip(i)
    + nuc_params.amplitude_strike(i) * nuc_params.amplitude_strike(i));
   pv_nuc_radial(i)    = nuc_params.radial(i);
   ```

   The 8 arrays must live for the lifetime of `pv_out` — allocate them at the same scope as `pv_local_slip` / `pv_local_slip_rate` (per the parity plan §"The 8 arrays MUST live for the lifetime of pv_out").

Interfaces

All interfaces are documented in the parity plan; this plan does NOT duplicate them.

Edge Cases to Handle

All edge cases are documented in the parity plan per sub-phase; this plan does NOT duplicate them.

Acceptance Criteria

- ☐ **All acceptance criteria from parity plan Phases 1-6** pass as written (the parity plan's own per-phase checklist).
- ☐ **TPV/BP5 byte-exact regression stays green** after every commit (parity plan §Risk Assessment §1's mitigation).
- ☐ **The SAFS production TOML default-OFF posture** is honoured: the EXAMPLE TOML and the runbook both render `paraview_volume = "off"` as the default and the user opts in per collection.

- ☐ **Schema doc is in sync** — every new TOML key added in Parity Phase 2 has a row in `spatial_friction_config_schema.md` `[output]` table.

Dependencies

- Depends on: Phase N (for `nuc_amplitude` / `nuc_radial_factor` values in Parity Phase 6).
- Required by: Phase Z (the production sbatch uses ParaView ZFP + regime cadence flags).

Phase Z: Production sbatch + smoke verifier

Goal

Produce two Frontera sbatches and a `fault.vtkhdf` smoke verifier so the first SAFS production run can be submitted, restarted, and post-checked end-to-end. After Phase Z, a user reading `spatial_workflow_safs_runbook_2026-05-18.md` §Step 7 has runnable commands.

Files to Create

- `miniapps/seas/jobs/safs/spatial_dyn_smoke_8N_400r_dev_2hr_safs.sbatch` — Frontera dev queue, 8 nodes × 50 ranks = 400 ranks, 2 hr wall, `--tfinal 0.5s`. Per `feedback_local_mpi_cores`: local rehearsal uses `mpirun -np 8`; the sbatch uses Frontera's 400 ranks for the 1000 m mesh.
- `miniapps/seas/jobs/safs/spatial_dyn_production_normal_24hr_safs.sbatch` — Frontera normal queue, 32 nodes × 50 ranks = 1600 ranks, 24 hr wall, `--tfinal 12s`, V1 checkpoint every 10000 steps. Node count derived **OFFLINE** (R-006 fix: shell scripts cannot call MFEM at submission time): implementer runs `seas_spatial_dyn_driver --dry-run --print-derived` on a single Frontera dev node FIRST, reads `num_global_elements = X` from the `[derived]` log, computes `nodes = ceil(X / (50 ranks/node · 2500 elem/rank))`, then bakes the result into the sbatch header with a comment citing the dry-run log line. The 2500-elem/rank target is empirical (from BP5/TPV104 scaling); revisit after the first production run if step time is dominated by MPI communication.
- `miniapps/seas/safs/project_7.0_alternative/spatial/code/scripts/verify_spatial_dyn_smoke_safs.py` — opens `<output_dir>/fault.vtkhdf`, asserts:
  - Max slip-rate in the rupture core (`r < 3 · max(radius_*)` from the TOML's nucleation block) ≥ 1e-3 m/s by `t ≥ T_nuc_s`.
  - No NaN in any field.
  - `σ_n > 0` everywhere.
  - V_max peaks then decays (no monotonic growth). Writes a 1-page text summary `verify_summary.txt` next to the `fault.vtkhdf` and exits 0/1 accordingly.

Files to Modify

- `miniapps/seas/Makefile` — add new umbrella targets (optional):

  `test-spatial-dyn-smoke-sbatch: jobs/safs/spatial_dyn_smoke_8N_400r_dev_2hr_safs.sbatch`

  (lint-only via `sbatch --test-only`; no actual submission from CI).

- `miniapps/seas/.gitignore` — confirm `safs/project_7.0_alternative/spatial/code/scripts/__pycache__/` is already ignored.

Interfaces

```
# Smoke (dev queue, 2 hr):
sbatch jobs/safs/spatial_dyn_smoke_8N_400r_dev_2hr_safs.sbatch

# Production (normal queue, 24 hr):
```

```
sbatch jobs/safs/spatial_dyn_production_normal_24hr_safs.sbatch

# Restart:
sbatch jobs/safs/spatial_dyn_production_normal_24hr_safs.sbatch --restart <cp>

# Post-run:
python miniapps/seas/safs/project_7.0_alternative/spatial/code/scripts/verify_spatial_dyn_smoke_safs.py \
    --fault-vtkhdf <output_dir>/fault.vtkhdf \
    --toml-config <config.toml>
```

Detailed Requirements

1. **Smoke sbatch structure** — mirror `jobs/safs/safs_smoke_8N_400r_dev.sbatch` header (queue, allocation, output paths). Tee `seas_spatial_dyn_driver` stdout and stderr to `<output_dir>/spatial_dyn_smoke.log`. The driver invocation:

   ```
   ibrun ./seas_spatial_dyn_driver \
       --config <CONFIG_TOML> \
       --mesh   ${SAFS_ROOT}/meshing/results/msh/safs_fault_box_nwcut_1000m_zgraded.msh \
       --no-sidecar-material \
       --tfinal 0.5s \
       --paraview \
       --paraview-fault-hdf5 \
       --paraview-fault-zfp-tol 1.0e-12 \
       --paraview-max-snapshots 200 \
       --print-derived \
       --output-dir ${SCRATCH}/safs_dyn_smoke_${SLURM_JOBID}
   ```

   The `--print-derived` flag combined with the actual time loop means the smoke is BOTH a pre-flight gate AND an end-to-end run.

2. **Production sbatch structure** — same as smoke, plus:

   ```
       --tfinal 12s \
       --checkpoint-every 10000 \
       --paraview-coseismic-dt 0.005 \
       --paraview-nucleation-dt 0.001
   ```

   Add a `RESTART_PREFIX` shell variable detected from `${SCRATCH}` so a second `sbatch` submission with the same `JOBNAME` automatically resumes from the most recent checkpoint:

   ```
   if [[ -f "${OUT}/cp_t000010000.h5" ]]; then
       RESTART_FLAG="--restart ${OUT}/cp"
   fi
   ```

   This mirrors the BP5 + TPV104 restart pattern from `jobs/bp5/bp5_v2_restart_*.sbatch`.

3. **Verifier `verify_spatial_dyn_smoke_safs.py`** — Python 3 stdlib + **third-party h5py** (R-005 fix: h5py is NOT in the Python stdlib). Run under the `pythonenv` conda env (per `CLAUDE.md` Environment Setup) which has h5py for the mesh/velocity/stress pipelines. At the top of the script, fail-fast with a helpful message if the import fails:

   ```
   try:
       import h5py
   except ImportError:
       sys.exit("verify_spatial_dyn_smoke_safs.py requires h5py.  "
                "Run under `conda activate pythonenv` (per "
                "miniapps/seas/CLAUDE.md Environment Setup).")
   ```

Reads `fault.vtkhdf`, the TOML config (for the nucleation centre + radii), and the run's `[derived] PASS/FAIL` line from the log. Checks:

- **A**: `slip_rate_max` across the dataset's rupture-core DOFs >= 1e-3 m/s by the snapshot whose t >= T_nuc_s.
- **B**: No NaN in `slip_rate`, `traction`, `sigma_n`, `mu_eff`.
- **C**: `sigma_n > 0` at every DOF in every snapshot.
- **D**: V_max time series is unimodal (one peak, then decay; allow small post-peak oscillation up to 10% of peak).
- **E**: The `[derived] PASS` line is present in the log. Output: `verify_summary.txt` with PASS/FAIL per check + per-check numeric values; exit code 0 if all pass, 1 otherwise. The 1-page format matches the existing `bp5_v2_restart_verify_*.py` template.

4. **Resource budget documentation** in each sbatch header:

```
# Resource budget (SAFS 1000m zgraded, constant material, tfinal=12s):
#   Mesh:       ~4M tets, ~120M DOFs (3D velocity-stress, order 1)
#   Per-rank:   ~2.5 GB peak (wave Q + DOFData + GodunovFlux)
#   Wall:       ~18-22 hr at cfl=0.5, dt~6e-5 s, 200k steps
#   Output:     fault.vtkhdf ~80 GB with ZFP tol 1e-12
#               cp files every 10k steps × ~5 MB/rank = ~8 GB total
#   Scratch:    100 GB recommended
```

Edge Cases to Handle

- **`mfem-dev` conda env not loaded in batch script** — the sbatch explicitly `module load`s the impi/intel modules used by the seas build (mirror `jobs/safs/safs_smoke_8N_400r_dev.sbatch`'s preamble).
- **`HDF5_PLUGIN_PATH` not set** — ZFP plugin lives at a Frontera-specific path; the sbatch sets it before `ibrun`. Document in the runbook §7.
- **Mid-run restart drops to a different node count** — V1 checkpoint format is per-rank, so restart MUST use the same `ibrun` rank count. Sbatch comment says "restart with `--ntasks` and `--nodes` matching the original submission".
- **`--print-derived` aborts during the smoke** — sbatch detects exit code != 0 and `exit 1` cleanly (no post-run verifier invocation).
- **`fault.vtkhdf` is empty** (zero snapshots written) — verifier prints `FAIL: fault.vtkhdf has zero cycles; check --paraview-max-snapshots and the actual tfinal vs T_nuc_s`. Exit 1.

Acceptance Criteria

- ☐ **T-Z01 — Smoke sbatch lint**: `sbatch --test-only jobs/safs/spatial_dyn_smoke_8N_400r_dev_2hr_safs.sbatch` exits 0.
- ☐ **T-Z02 — Production sbatch lint**: same for production.
- ☐ **T-Z03 — Verifier runs on a committed reference**: a small reference `fault.vtkhdf` checked into `safs/project_7.0_alternative/spatial/code/scripts/tests/fixtures/` passes the verifier (PASS exit 0). The fixture is a tiny inline-mesh smoke produced by the unit test runner; ≤ 5 MB.
- ☐ **T-Z04 — Verifier catches a deliberate failure**: a second fixture with `slip_rate_max = 0` (no nucleation) fails check A; verifier writes `FAIL: A (slip_rate_max=0)` and exits 1.

Dependencies

- Depends on: Phase N (resolver), Phase D (`--print-derived`), Parity Phases 1-6 (ParaView output).
- Required by: nothing.

Testing Strategy

Phase	Test target	What it proves
N	`seas_test_spatial_nucleation` (12 tests T-N01..T-N12)	Math correctness of smoothStep + Gaussian + accumulator + iterator callback hook
N	`seas_test_spatial_friction_resolver` (existing minus F-4/F-5/T-22)	Removal of obsolete code doesn't regress the remaining resolver tests
N	`make test-tpv104`, `make test-tpv205`, `make test-bp5-smoke`, `make test-bp5-integration`	TPV/BP5 byte-exact regression contract (the headline safety gate)
N	`make test-spatial-dyn-driver` (`--dry-run` smoke)	Driver builds + dry-runs end-to-end with the new nucleation
D	New `seas_test_spatial_print_derived` (T-D01..T-D07)	Pre-flight gate aborts on supercritical / insufficient setups; passes on well-posed setups
Parity	All parity plan T-30..T-42 (already enumerated in parity plan)	Full ParaView paraview-compaction feature parity
Parity	`seas_test_spatial_friction_config` extended (T-24..T-28 from parity plan §Phase 2 acceptance)	OutputSpec extension + default flip
Z	T-Z01..T-Z04	Sbatch lints + verifier round-trips
Cross	`make test` aggregate	Nothing in the project regressed

Risk Assessment

High-confidence (low risk)

- **Phase N temporal helpers** — `SmoothStep` / `SmoothStepIncrement` are copied verbatim from `tpv104_nucleation.hpp` (rename only). The TPV104 tests already exercise the math (`T_NUC_TPV104_R3-007` etc); the new unit tests mirror them.
- **Phase N accumulator pattern** — `ApplyGradualOverstressIncrement` mirrors `ApplyNucleationIncremental_TPV104` line-for-line with the only delta being "writes both tau1_nuc and tau2_nuc" instead of "writes only tau2_nuc". The TPV104 production runs have validated the pattern at scale.
- **Phase N dead-code removal** — the obsolete `Overstress` resolver is already a stub (aborts at runtime); deleting it cannot regress any passing code path.

Medium-risk

- **`Tpv205SubStepIterator` additive overload** — splicing a callback invocation into the inner sub-step loop requires copying the entire `AdvanceWithSubStepStates` body. The risk is that the body has subtle side effects between sub-steps that the copy misses. Mitigation: keep the original overload, route it through the new overload with a no-op callback, and assert via `T_TPV205_ITERATOR_OVERLOAD_PARITY` (T-N12 acceptance) that the original behaviour is bit-identical when `nuc_callback = noop`.
- **Equilibrium-gate threshold tuning (Phase D §2-3)** — the 90% nucleation overshoot threshold and the `outside_safety_factor = 3.0` Gaussian support threshold are heuristics. If too tight, the gate aborts on well-posed SAFS configs that the user wanted to run; if too loose, the gate passes ill-posed configs that then waste Frontera time. Mitigation: emit WARNING (not ABORT) in the 0.9-1.0 band per item 2; expose `SEAS_SKIP_EQUILIBRIUM_GATE` env var per item 3 edge case; document both thresholds in the runbook so users can override.
- **`AdvanceADERWithSubStep_Spatial` signature extension** — the wrapper is private to the driver, but if any future test or Phase R work links against it, the signature change is a breaking edit. Mitigation: it's a `static` free function inside the `spatial_dyn_driver.cpp` anonymous namespace; no external linkage.

- **Default flip from `paraview_volume = "hdf5"` to `"off"` (Parity Phase 2)** is user-visible. Mitigation: see parity plan §Risk Assessment §3 — search the SAFS dataset directory for any TOML that omitted `paraview_volume` (relying on the old default) and update each to spell out `"hdf5"` explicitly BEFORE the flip lands.

Low-confidence (need investigation before implementing)

- **`dof_basis` row layout** — Phase N requires the per-DOF basis-row block to be `[n_row; t1_row; t2_row]` (one 3×3 sub-block per DOF, indexed by `i`). The implementer MUST verify the exact layout by reading `BuildPerDOFFaultTables` in `dynamic/spatial_setup.hpp` AND by writing a tiny test (T-N09 implicitly does this — if the centre-DOF amplitude check fails, the layout is the suspect).
- **Tpv205 callback overload reaching all sub-step paths** — `AdvanceADERWithSubStep_Spatial` calls `iterator.AdvanceWithSubStepStates` but the iterator also exposes `Advance(...)`. Audit `grep -rn 'iterator\.Advance' miniapps/seas/`: if `Advance` is called from any non-test path the callback hook must be added to BOTH overloads. (Per the iterator hpp docstring §R-011, the production driver uses `AdvanceWithSubStep_States`; `Advance` is the time-averaged $\bar{Q}$ path for API symmetry. Verify before deciding to skip the `Advance` overload.)
- **`Tpv205SubStepIterator::StepOneQP_` writes between sub-steps** — if the per-QP step writes back to `dof_data` between sub-steps, and the nucleation callback is called BEFORE the per-QP loop, the ordering is: nucleation → friction solve → write-back. This matches TPV104's ordering at `tpv104_substep_iterator.cpp:295` (nucleation call) then L300+ (per-QP loop). Confirm the LSW path's `StepOneQP_` does not read `tau{1,2}_nuc` BEFORE the friction solve — if it does (e.g. via `flux_.ComputeStageState`), the nucleation perturbation must be visible to that read. The TPV104 ordering already enforces this; verify by reading `tpv205_substep_iterator.cpp::StepOneQP_`.

Known tricky areas in existing code

- **`mfem::Geometries.GetCenter(gtype)` may not be the centroid** for curved tets — see the comment block at `spatial_friction.cpp:1017-1019` about the "true reference centroid" stop-gap (`R-111`). For Phase D's `cp` value used in σ_1 azimuth (which uses scalar `cp` not per-element), this is a non-issue. For Phase R's per-element heterogeneous CFL it matters; not in scope here.
- **`FaultGeometry::NumZeroNormalFallbacks()`** can be non-zero on the SAFS curvilinear mesh (R-001 in the parent plan). Phase D Detailed Req. 7 surfaces this to the user; do not silently swallow.
- **V1 checkpoint `driver_tag` field** — Phase 5b already added this; the driver writes `"spatial_dyn"`. Phase N's removal of the `LSW_ForcedRupture` dispatch from the spatial path does NOT change the checkpoint schema (no fields added/removed); existing `spatial_dyn`-tagged checkpoints from Phase 4 will restart cleanly into the post-Phase-N driver. Verified by T-N restart smoke (manual; not in the automated test list).

Commit cadence

This plan is intentionally structured as **four independently mergeable commits** with the TPV/BP5 byte-exact regression as the gate between each.

Commit	Phase(s)	Bisect surface caught if this commit fails	Wall-time est.
#1	Phase N (nucleation)	Math errors in `SmoothStep` / `GaussianFactor` / accumulator (12 unit tests); regressions in the obsolete-code removal (TPV/BP5 byte-exact); ordering bug in the iterator callback hook (T-N12)	2-3 days
#2	Phase D (print-derived)	False-positive aborts on well-posed setups (T-D01); false-negative passes on supercritical setups (T-D02); σ_1 azimuth math (T-D06)	1 day

Commit	Phase(s)	Bisect surface caught if this commit fails	Wall-time est.
#3	Parity Phases 1-6	Drift in TPV/BP5 byte-exact paths from `OutputSpec` extension + `pv_out` construction logic; default-flip user-visible regressions; SAFS fault static-fields layout bugs in `SetFaultParamsSpatial`	3-5 days
#4	Phase Z (sbatch + verifier)	Operational issues only (queue config, plugin paths, restart prefix detection) — no physics	1-2 days

Strict gate: each commit requires the TPV/BP5 regression suite (`make test-tpv104`, `make test-tpv205`, `make test-bp5-smoke`, `make test-bp5-integration`) to be bit-identical to its parent. If any of those fails after a commit, REVERT and bisect; do NOT advance to the next commit. This is the same gate Phase H Stage 1's `T-PHASEH-SCALAR-PARITY` enforced and is the safety net for the whole plan.

Commit message templates (CLAUDE.md feedback_complete_sign_sites): each commit's message must enumerate every file modified AND state explicitly which files were AUDITED but not modified (e.g., `dynamic/tpv104_*.hpp, drivers/tpv104_driver.cpp, drivers/tpv205_driver.cpp — audited, no change`).

Minimum-viable critical path

```
Commit #1  Phase N   (gradual_overstress nucleation)              ~2-3 days
Commit #2  Phase D   (--print-derived + equilibrium gate)         ~1 day
           ── Local smoke: mpirun -np 8 ... --dry-run --print-derived ──
Commit #3  Parity Phases 1-6 (full ParaView feature surface)      ~3-5 days
          ── Local smoke: mpirun -np 8 ... --tfinal 0.5s; inspect fault.vtkhdf in ParaView ──
Commit #4  Phase Z   (Frontera sbatch + verifier)                 ~1-2 days
           ── Frontera smoke: sbatch ...smoke...sbatch (2hr dev queue) ──
           ── Frontera production: sbatch ...production...sbatch (24hr) ──

Total implementation: ~7-11 days
+ 1 Frontera smoke wait (~hours)
+ 1 Frontera production wait (~24hr)
```

After Commit #1 + #2, the driver can run end-to-end with a sensible nucleation and a hard pre-flight gate; ParaView output is still the existing 5 fault projections only. After Commit #3, the user can see the heterogeneous LSW + nucleation amplitudes and the volume velocity in ParaView. After Commit #4, the run is production-ready on Frontera.

If the user wants the smallest possible path to a *runnable* SAFS smoke on Frontera (skipping full ParaView parity), Commit #1 + #2 + a minimal sbatch suffice; the secondary collection (Parity Phase 4) and volume velocity (Parity Phase 5) are deferrable to a later branch.

Out of scope (deferred to a separate branch, per user decision)

The following items are EXPLICITLY OUT OF SCOPE for this plan and MUST NOT be implemented as part of any commit in this plan:

- **Phase H Stage 2** (per-element flux dispatch in `wave_operator.inl`). `MaterialField::Mode::Coefficient` continues to abort at the `WaveOperator(MaterialField)` ctor; the spatial driver's `--no-sidecar-material` requirement at `spatial_dyn_driver.cpp:749-757` stays in place.
- **Phase H.5** (cross-rank bi-material MPI exchange for `shared_face_neighbour_material_`).
- **Phase R** (exact bi-material Riemann solver: `BimaterialFlux` class, `BimaterialFluxMode` enum, per-face pre-computed flux matrices, layered-medium analytic R/T verification). See `safs/project_7.0_alternative/document/PLAN_phas`

- **Spatial Phase 2** (`FaultFaceFlux::InitializeImpedancesPerQP`). The per-fault-QP `η_p / η_s` impedance setter is a no-op under constant bulk material; the existing scalar `InitializeImpedancesFromMaterial` is correct.
- **CVM sidecar consumption** (CVMH / CVM-S 4.26.M01 / multiscale_statewise). The sidecars at `velocity/results/<model>/velocity_safs.h5` are produced by the Python pipeline but NOT consumed by the spatial driver under the constant-material assumption.
- **Two-CVM comparison workflow** (R-109 in the dynamic-rupture review). Deferred with the heterogeneous-material stack.
- **TPV26/27 forced-rupture nucleation** (`NucleationKind::StrengthReduction`, `LSW_ForcedRupture` dispatch arm, `T_forced_rupture` / `t0_decay_forced` DOFData fields). Removed from the spatial driver; native TPV* drivers keep their own paths.
- **TPV205-style instantaneous overstress nucleation** (the partially- scaffolded `NucleationKind::Overstress` / `OverstressSpec` / `ResolveOverstress`). Removed entirely.
- **Spatial Phase 1 setter formalization** (`FaultFaceFlux::InitializeFromSpatialStress` setter + `dynamic/spatial_dyn_fault_setup.{hpp,cpp}` glue). The inline per-DOF write in the driver works correctly today; the refactor adds test coverage but no correctness.
- **Spatial Phase 5c helper** (`ComputePerDOFCoordsAndBasisFromWave`). Inline walk in the driver works correctly today.
- **`Tpv205SubStepIterator::SetForcedRuptureMode` toggle** — irrelevant now that the spatial driver does not use `LSW_ForcedRupture`.
- **Cross-verification against an external code** (Tandem, SeisSol) on a SAFS-equivalent setup. The runbook references this as the "strongest external accuracy claim" but the cross-verification campaign is its own multi-week effort.

Appendix: greppable TODO catalog

Phase N introduces these `SPATIAL-DYN-FIRST-RUN R-XXX TODO` markers, each with a clear scope and follow-up owner:

- `R-001` — Document zero-normal-fallback handling for `GaussianFactorFaceLocal`: if a fault DOF hit the fallback, `GaussianFactorFaceLocal` will produce a non-zero but incorrectly rotated value. Currently surfaces as a `[derived] WARNING` in Phase D §7; production remediation is to fix `FaultGeometry`'s basis computation on SAFS curvilinear faults.
- `R-002` — `--print-derived` σ_1 azimuth computation for sidecar stress mode. Currently emits a TBD line; production needs a per-DOF azimuth histogram. Phase D §4 documents the deferral.
- `R-003` — V1 checkpoint test for restart-into-Phase-N driver from a Phase 4 / 5a / 5b spatial_dyn checkpoint. Currently a manual smoke; production needs an automated round-trip test. Phase Z Acceptance §"manual restart smoke" documents the deferral.
- `R-004` — `Tpv205SubStepIterator::Advance` (the time-averaged $\bar{Q}$ overload, distinct from `AdvanceWithSubStepStates`): if any non-test path ever uses it, the callback hook must be plumbed into it too. See Risk Assessment "Tpv205 callback overload reaching all sub-step paths". Currently only the `AdvanceWithSubStepStates` path is plumbed.